



Tent

Fully Test-Time Adaptation by Entropy Minimization

Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen & Trevor Darrell

ICLR 2021 (Spotlight) · [arXiv:2006.10726](https://arxiv.org/abs/2006.10726)

UC Berkeley and Adobe Research

Adapting at test time

Why this paper matters



A one-line loss that defined a subfield

< 1%

of model parameters are
updated at test time

1

gradient per test point —
no extra forward passes

44.0%

ImageNet-C error, against
50.2% for robust training

0

source data, labels, or
changes to training

● The claim

A deployed model meets data it was not trained for, and accuracy suffers. It has only its own parameters and the unlabelled test stream. Minimise the entropy of its own predictions, updating only the channel-wise scales and shifts in its normalisation layers, and error falls — in one epoch, without altering training at all.

● The authors' own framing of the contribution

Entropy minimisation was already well established as a regulariser for domain adaptation, semi-supervised learning and few-shot learning. The contribution is to exhibit entropy as the sole loss for fully test-time adaptation — not a new objective, but a demonstration that one familiar objective suffices on its own.

1

The setting

What 'fully test-time' excludes, and why that matters



Four adaptation settings



They differ by what data is available, and therefore what loss is possible

Setting	Source data	Target data	Train loss	Test loss
Fine-tuning	—	x_t, y_t	$L(x_t, y_t)$	—
Domain adaptation	x_s, y_s	x_t	$L(x_s, y_s) + L(x_s, x_t)$	—
Test-time training	x_s, y_s	x_t	$L(x_s, y_s) + L(x_s)$	$L(x_t)$
Fully test-time adaptation	—	x_t	— (training unchanged)	$L(x_t)$

- **The row that is new**

Test-time training also adapts during testing, but it must alter training first — jointly optimising a supervised loss and a self-supervised proxy on source so the parameters are shared and compatible with adaptation later.

Fully test-time adaptation is independent of the training data and training loss given the parameters. In the authors' phrase: if the model can be run, it can be adapted.

Why this setting is the realistic one



Three motivations from the introduction

● Availability

A model might be distributed without its source data — for bandwidth, for privacy, or because the data is commercially valuable. The weights ship; the training set does not.

● Efficiency

Even where the source data exists, it may not be computationally practical to reprocess it during testing. ImageNet is over a million images; the adaptation window is an inference call.

● Accuracy

A model might simply be too inaccurate without adaptation to serve its purpose. Inference has to go on before new data can be collected and annotated.

● Why this matters for the second half

All three motivations apply with extra force in speech emotion recognition: the source corpora are often licence-restricted, utterances arrive one at a time, and we have spent the whole term documenting how badly emotion models transfer across corpora.

Emo-TTA takes the setting and tightens it further — to a single sample, with no gradients at all.

2

The method

Why entropy, which parameters, and the algorithm



Why entropy



Two empirical observations, and one argument from principle

● Entropy tracks error

On corrupted CIFAR-100, predictions with lower entropy have lower error rates.

So certainty can serve as supervision during testing — the model's own confidence stands in for the labels it does not have.

● Entropy tracks shift

More corruption causes more loss and more entropy, with a strong rank correlation as the severity level rises.

So entropy can estimate the degree of shift without training data or labels — it is a signal that something has changed.

● And it is task-defined

Self-supervised proxy tasks — rotation prediction, context prediction, colourisation — are not directly related to the supervised task, so progress on the proxy can interfere with the task.

Entropy is a measure of the predictions themselves, so it cannot be off-task.

● The third argument is the strongest

Test-time training needs a proxy that is, in its authors' words, both well-defined and non-trivial in the new domain — which requires choosing the proxy, designing its head, and balancing two losses.

Entropy needs none of that. It is defined by the task the model was already trained for.

The objective



Shannon entropy, and why it needs a batch

● The loss

$H(\hat{y}) = - \sum_c p(\hat{y}_c) \log p(\hat{y}_c)$, the Shannon entropy of the predicted distribution over classes.

It depends only on predictions, not annotations — which is what makes it available at test time.

● The degenerate solution

Optimising a single prediction has a trivial solution: assign all probability to the most probable class. Entropy goes to zero and nothing is learned.

The fix is to jointly optimise batched predictions over parameters shared across the batch — so the collapse would have to be consistent across many different inputs.

● This constraint is the one the second talk removes

Tent's entropy objective structurally requires a batch. That is fine for image classification, where test data arrives in bulk, and awkward for streaming speech, where an utterance arrives alone.

The paper names this as an open problem in its own discussion: entropy needs batches for optimisation, and cannot update episodically on one point at a time.

Which parameters to update



The argument against updating the model

● Why not all of θ

Three objections, all stated in the paper:

θ is the only representation of the training data in this setting, so altering it could cause the model to diverge from its training.

The function is non-linear and θ is high-dimensional, making optimisation too sensitive and inefficient for test-time use.

● What is updated instead

Feature modulations that are linear — scales and shifts — and low-dimensional — channel-wise.

Normalisation: estimate μ and σ from the target data, discarding the source statistics entirely.

Transformation: optimise the affine parameters γ and β by the gradient of the entropy.

● Note the division of labour

The statistics μ , σ are estimated from the data. The parameters γ , β are *optimised by the loss*. Two different mechanisms in the same layer, and the ablation shows both are needed.

In implementation this is simply repurposing the normalisation layers the source model already has, which is why Tent works on any architecture with them and why it is about ten lines of code.

The algorithm



Three steps, and one scheduling detail

- 1 Initialisation**

Collect the affine parameters $\{\gamma, \beta\}$ for every normalisation layer and channel. Fix everything else. Discard the source normalisation statistics entirely.
- 2 Iteration**

Per batch: estimate normalisation statistics layer by layer during the forward pass; update γ and β by the gradient of the prediction entropy during the backward pass.
- 3 The scheduling detail**

The transformation update follows the prediction for the current batch, so it only affects the next batch. This costs one gradient per point and no extra forward passes — which is why it is the default.
- 4 Termination**

Online: never — iteration continues as long as test data arrives. Offline: update first, then repeat inference, optionally for multiple epochs.

3

Results

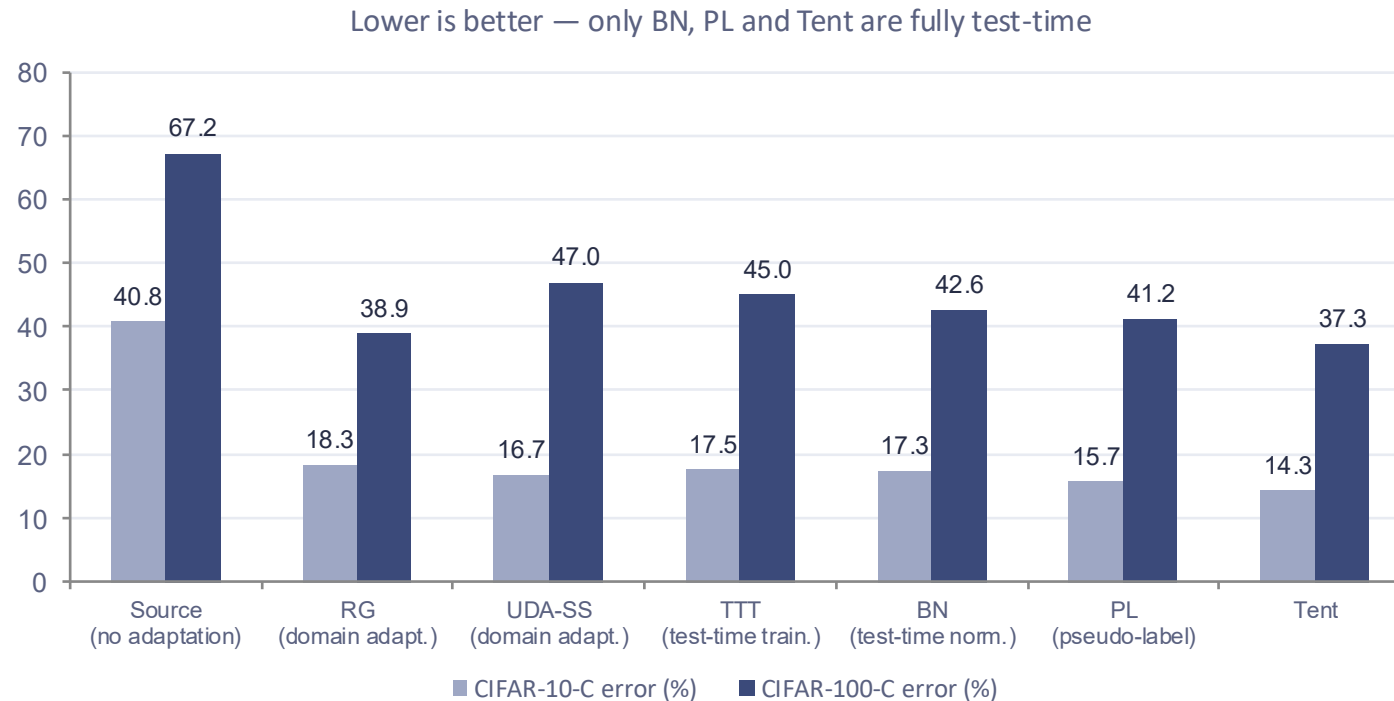
Corruptions, digits, segmentation, VisDA



Corruption robustness



CIFAR-10-C and CIFAR-100-C at the highest severity



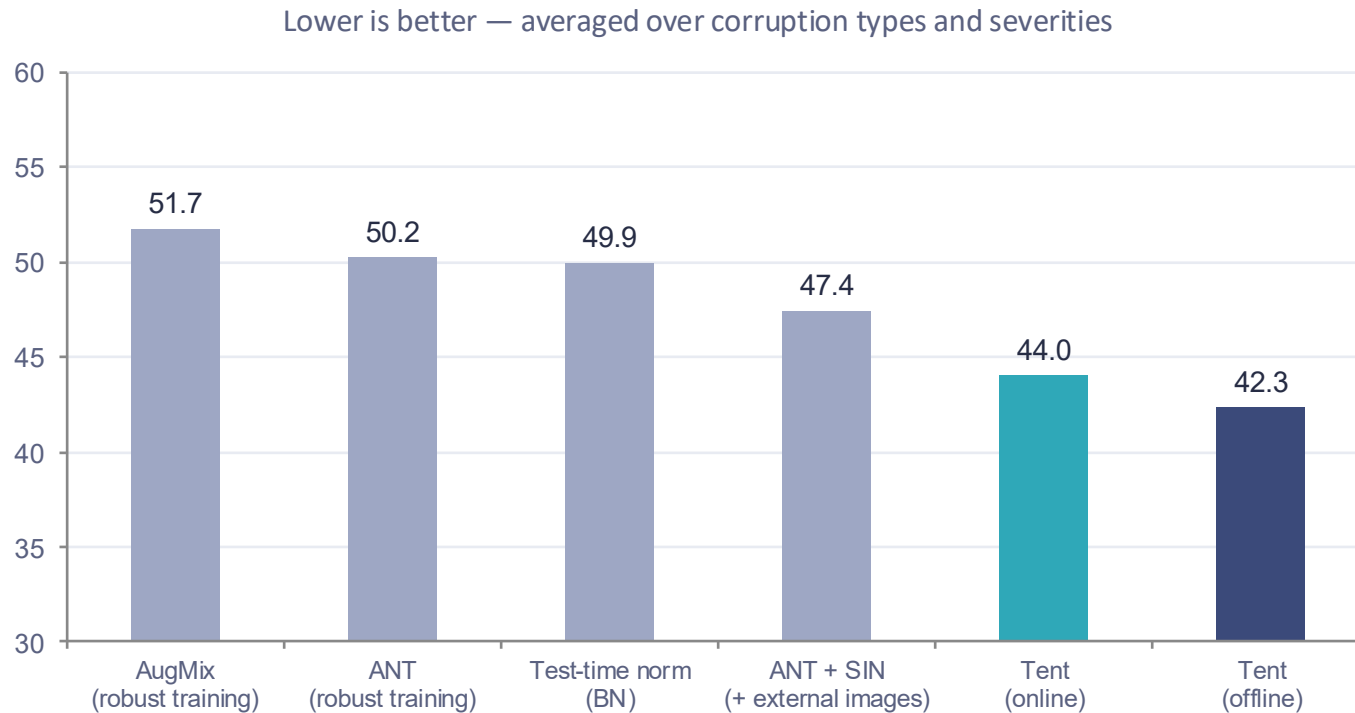
- Tent has the lowest error on both, beating methods that use the source data and several epochs of joint optimisation.
- The three middle methods all require more: RG and UDA-SS train jointly on source and target; TTT alters training with a self-supervised proxy.
- Test-time normalisation alone already recovers most of the gap — 67.2 to 42.6 on CIFAR-100-C. Tent's gradient adds a further 5.3.
- Pseudo-labelling, the other fully test-time baseline, sits between them.

The striking comparison is with the middle three: less data, less optimisation, and no change to training — and lower error.

ImageNet-C



A new state of the art without altering training



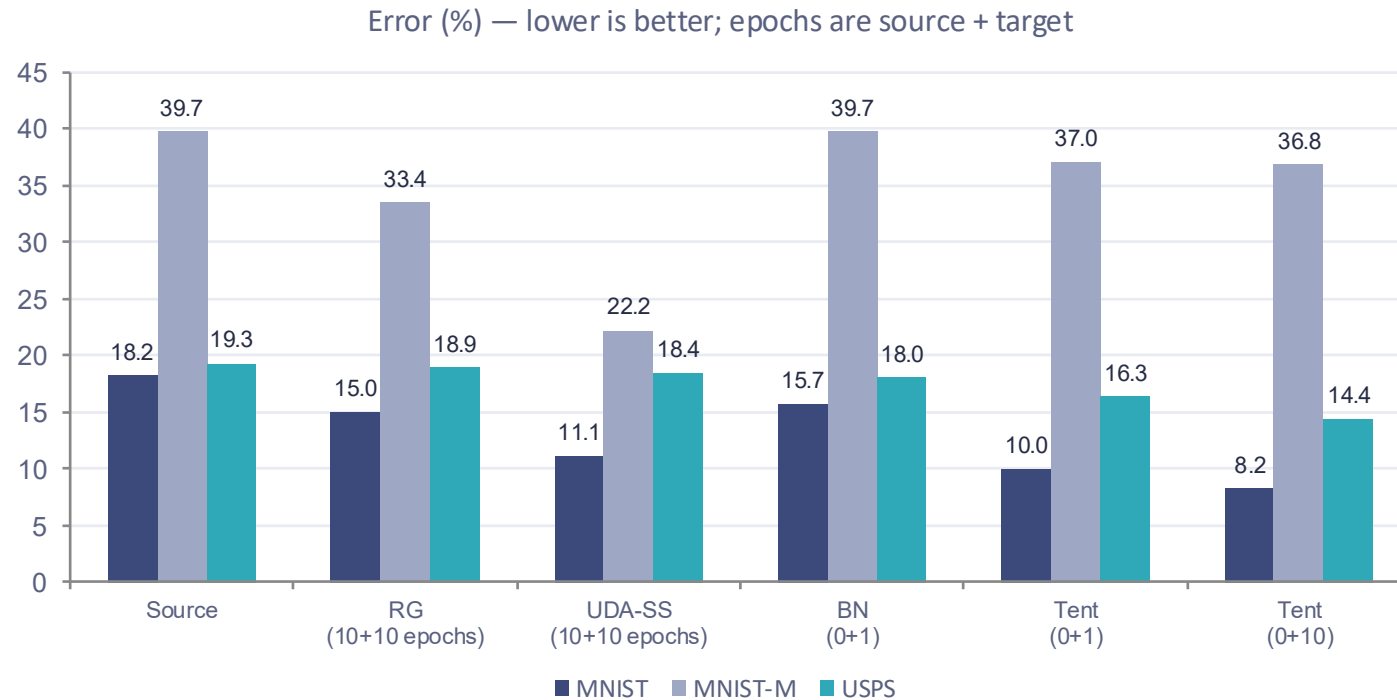
- The prior state of the art, adversarial noise training, spends extra epochs on the full ImageNet training set. Tent spends one gradient per test point.
- Combining robust training with stylisation from external images reaches 47.4; Tent reaches 44.0 online and 42.3 offline, with neither.
- A relative 18% improvement over test-time normalisation among fully test-time methods.
- Tent improves on ANT for every corruption type except noise — the one ANT is explicitly trained on.

The headline that carried the paper: you can beat training-time robustness methods by adapting at inference, for less compute than they spend.

Source-free domain adaptation



SVHN to MNIST, MNIST-M and USPS



- Tent always beats both the source model and test-time normalisation, and wins outright on two of the three shifts.
- On MNIST-M, self-supervised domain adaptation is far ahead — 22.2 against 37.0 — because it can use the source data jointly.
- The efficiency argument: RG and UDA-SS also process SVHN train, roughly seven times the size of the target, for ten epochs. Tent adapts with about eighty times less computation.
- With more updates Tent keeps improving: 8.2 at ten epochs and 6.5 at a hundred.

The paper's own conclusion: unsupervised domain adaptation remains necessary for the highest accuracy and the harder shifts.

Beyond classification



Segmentation, VisDA, and other architectures

- **Semantic segmentation**

Simulation-to-real from GTA to Cityscapes with HRNet-W18. Intersection-over-union: source 28.8%, test-time normalisation 31.4%, Tent 35.8%. Adapting to a single image — treating it as a batch of pixels — reaches 36.4% in ten iterations.

- **VisDA-C**

Synthetic-to-real object recognition over twelve classes. Source error 56.1%, Tent 45.6%, and 39.6% when updating all layers except the final classifier as SHOT does. Offline source-free methods reach lower error with more computation and tuning.

- **Other architectures**

On CIFAR-100-C, Tent transfers without tuning to self-attention networks (55.3 $\rightarrow$ 36.7 and 48.0 $\rightarrow$ 29.2) and to a deep equilibrium model (53.3 $\rightarrow$ 41.7).

- **The generality claim**

Tent needs a model that is trained, probabilistic and differentiable, with normalisation layers. Typical supervised deep networks satisfy all of these.

4

Analysis and limits

Where the premise is thin, and where the method fails



Does entropy actually track error?



The evidence for the central premise

● What was measured

For all 75 combinations of corruption type and level on CIFAR-100-C, the paper plots the change in entropy against the change in task loss after adaptation.

Both axes normalised by the maximum entropy of a prediction and clipped. Most points have lower entropy and lower error after adaptation.

● The number

The rank correlation coefficient between change in entropy and change in loss is 0.22.

The paper reports this in a figure caption, alongside 'note the dark diagonal', and does not comment on its magnitude.

The caption also notes a case where Tent decreases entropy but increases loss.

● How to read that

A rank correlation of 0.22 is a weak positive relationship. It is consistent with the premise — reducing entropy usually reduces error — and it is a long way from the premise being reliable.

That is the honest version of the paper's thesis: entropy is a usable proxy for error, not a faithful one. Everything downstream, including the failure case two slides from now, follows from that gap.

Which parameters, tested



The ablation that justifies the design

- **Not updating normalisation**

Errors increase, and the method can fail to improve over test-time normalisation or pseudo-labelling. The statistics estimation is doing real work.

- **Not updating transformation**

Reduces the method to test-time normalisation — a strong baseline, but 5.9 points worse on ImageNet-C.

- **Updating only the last layer**

Can improve, but then degrades with further optimisation. Capacity in the wrong place is unstable.

- **Updating the full model θ**

Never improves over the unadapted source model. The paper's sharpest negative result, and the justification for restricting to modulation.

- **Generalisation across points**

Adapting on target train and evaluating on target test still helps — CIFAR-100-C goes from 37.3 to 34.2. So what is learned is not point-specific.

Where Tent fails



Three honest limitations from the paper

● A hard shift

MNIST to SVHN, with a source-only error of 71.3%.

UDA-SS reaches 38.7% and DIRT-T 45.5%.

Tent *fails on this shift and increases error to 79.8%* — worse than doing nothing.

● Natural shifts

CIFAR-10.1 and ImageNetV2 were built by reproducing the original dataset collection procedures, so they entail natural but unknown shifts.

Error is higher on both, indicating shift is present — and Tent does not improve generalisation.

● The batch requirement

Entropy needs batches for optimisation and cannot update episodically on one point at a time.

Test-time training can do so, but only with the right proxy task.

● Why the first failure is instructive

When the source model is already badly wrong, its confident predictions are confidently wrong — and minimising entropy makes it more confident in exactly those errors.

Entropy minimisation amplifies whatever the model already believes. That is a feature when it is roughly right and a failure mode when it is not.

The discussion section as a research agenda



Three questions the paper poses and does not answer

● Shifts

Tent handles corruptions, simple appearance changes and simulation-to-real. It does not help on natural shifts, and adversarial shifts are untested. What other real-world shifts exist, and which does test-time adaptation address?

● Parameters

Modulation is reliable but limited. The issue is identifying parameters that are both expressive and reliable, and this may interact with the choice of loss. Adapting the input rather than the model is floated as a more general alternative.

● Losses

Is there an effective loss for general but episodic optimisation — one point at a time? And could a loss defined on the representation rather than the prediction avoid recomputing the model for every update?

● Calibration

The final suggestion: entropy may interact with calibration, since better uncertainty estimation could drive better adaptation.

5

Appraisal and legacy

What Tent started



Critical appraisal



What the design buys, and what it costs

● Strengths

A clean definition of a setting the field needed, with a table that made the distinctions legible.

Radical simplicity: one loss, under 1% of parameters, one gradient per point, about ten lines of code on top of existing normalisation layers.

Breadth of evidence: corruptions at two scales, three digit shifts, segmentation, VisDA, and three architectures including non-convolutional ones.

Honest failure reporting — MNIST-to-SVHN, natural shifts, and the cases where domain adaptation still wins.

A discussion section that reads as a research agenda, and largely became one.

Code released and widely reproduced.

● Limitations

The central premise rests on a rank correlation of 0.22 between entropy change and loss change.

Entropy minimisation amplifies existing beliefs, so it entrenches errors under severe shift.

The batch requirement is structural, not incidental, and rules out single-sample streaming.

Depends on normalisation layers — the method has no obvious form for architectures without them.

Hyperparameters are set by following the source model's training recipe, and later work showed test-time adaptation is unusually sensitive to this choice with no labelled data to tune on.

No treatment of what happens over long streams, where errors can compound.

Legacy



What Tent started, and the bridge to the second half

- **The TTA subfield**

Fully test-time adaptation became a standard setting with its own benchmarks, and Tent is the baseline every subsequent method reports against.

- **Continual and lifelong TTA**

Work on long test streams found that Tent accumulates error and eventually collapses — the confidence-amplification mechanism, played out over time — prompting methods with resets, teachers, and stability constraints.

- **Beyond vision**

Test-time adaptation moved into speech recognition and, more slowly, into speech emotion recognition. A 2026 systematic evaluation across eleven TTA methods and three SER tasks concluded that backpropagation-free methods are the most promising.

- **The gradient-free turn**

Methods that avoid backpropagation entirely — using retrieval, caching, or closed-form statistical updates — became the practical answer for streaming and privacy-sensitive settings. That is where the second talk sits.

Take-aways



Four things to carry into the second half

1 Confidence can substitute for labels

Entropy is available at test time, defined by the task, and needs no proxy design. That combination is why one familiar objective was enough to define a setting.

2 It amplifies, it does not correct

Entropy minimisation sharpens existing decision boundaries. Helpful when they are roughly right; it entrenches error when they are not, as MNIST-to-SVHN shows.

3 Restricted capacity is what keeps it stable

Updating the full model never improves. Under 1% of parameters, channel-wise and affine — the same restricted-budget logic as prompt tuning.

4 Check how strong the proxy correlation is

The premise rests on a rank correlation of 0.22. A method can work reliably on a loose mechanism, but you should know which you have.

Questions for discussion



1

The rank correlation between entropy change and loss change is 0.22. Is that enough to justify entropy as the sole objective — and what would a stronger proxy look like?

2

Tent fails on MNIST-to-SVHN, raising error from 71.3% to 79.8%. Could a method detect that it is about to make things worse, using only unlabelled test data?

3

Updating the full model never improves, but updating under 1% of it does. Is the lesson about entropy specifically, or about unsupervised test-time objectives in general?

4

The batch requirement is structural. What would an episodic, single-sample test-time objective have to look like — and what does that rule out?

5

Tent has no labelled data and yet needs a learning rate. How should test-time adaptation methods be evaluated when their hyperparameters cannot honestly be tuned?